

Gold and silver credit, settled on proofs.

A lending and dealing desk on Creditcoin where Chainlink price and Proof-of-Reserve feeds are carried from Ethereum as Merkle inclusion proofs and verified by the block-prover precompile inside the same transaction that acts on them.

◆ ABSTRACT

Tokenised real-world assets inherit a trust problem their collateral does not have. The metal sits in an audited vault and its price is published by an independent feed, but the chain that lends against them can read neither. Every production answer routes both facts through an operator who writes them onto the lending chain, and that operator — not the vault, not the auditor, not the feed — becomes the thing a lender actually trusts.

MEU Exchange removes the writer. Four independently operated Chainlink feeds on Ethereum mainnet — two metal prices, two Proof-of-Reserve attestations — and a collateral escrow on Ethereum Sepolia are carried to Creditcoin by the Attestcoin Protocol as Merkle inclusion proofs bound to attested source blocks. Creditcoin's block-prover precompile verifies each proof natively, synchronously, in the same transaction as the business logic that consumes it. The contracts are written so that a missing or stale proof is not a degraded display but a revert: issuance beyond proved reserves, a draw before the pledge is proved, and a fill against an unrefreshed round each fail with a named error.

This paper describes the venue, the transport, the contract model, the listing path and the risk parameters, and then states plainly what the construction does not achieve. It proves provenance, not truth: that a named feed published a number, not that the number is correct. The auditor remains trusted. That boundary is argued here as a feature of an honest design rather than smoothed over, because a proof system oversold is worse than one honestly bounded.

◆ CONTENTS

1 The trust gap in on-chain RWA credit

2 Creditcoin as the venue

3 The Attestcoin readability model

4 System architecture

5 The proof flow, end to end

6 The contract model

7 Listing a real-world asset

8 Risk parameters

9 Evidence from the testnet deployment

10 What this design does not prove

11 Deployment reference

The trust gap in on-chain RWA credit

A desk that lends against bullion has to answer three questions, and a blockchain is the wrong instrument for all three. **Is the collateral really escrowed?** The escrow lives on whichever chain the tokenised metal was issued on. **What is the metal worth right now?** The reference price is published by a feed operating on a chain chosen for liquidity, not for lending. **Are the issuer's reserves real?** That is an auditor's finding, published as a Proof-of-Reserve feed somewhere else again.

Each answer exists, on a chain, in a form a contract could in principle check. None of them is visible from the chain that needs them. The standard remedy is a relay: an operator observes the source chain and writes the observed value into a contract on the destination chain. The value is now readable, and the guarantee has quietly changed. The lender no longer trusts the auditor and the feed; they trust the operator's willingness and ability to report both faithfully, and the compromise of one key now moves a book.

Displaying a proof is not the same as requiring one

A weaker version of the same gap appears in systems that do carry attested data but treat it as decoration. Attested figures are rendered in an interface while the contract's state transitions run on separately written values, so the proof is evidence offered to a human rather than a precondition enforced by code. The distinction is observable: if the proof is missing or stale, does the system show a warning, or does the transaction fail?

THE DESIGN RULE

Every state transition that moves value in this system is gated on a proof verified in the same transaction. Where a proof is absent, stale, replayed, or emitted by an unbound address, the call reverts with a named error rather than proceeding on a lesser guarantee. Section 9 shows each of those refusals executed on a live testnet, paired with the transaction that then succeeded.

Three properties follow from that rule, and they are the claims this paper defends. Supply cannot exceed reserves that a Chainlink Proof-of-Reserve feed has published and this chain has verified. Credit cannot be drawn against collateral whose escrow has not been proved from the chain holding it. A trade cannot fill at a price no one has re-proved inside the staleness bound.

Creditcoin as the venue

Creditcoin is an EVM-compatible layer-1 blockchain built with the Polkadot SDK, whose documentation describes it as the evolution of an earlier Substrate-based credit protocol into a chain where "developers can build credit applications using familiar Ethereum tools." [1] The name now refers specifically to the EVM-compatible network; the original Substrate protocol is documented separately as CC Enterprise.

Consensus and execution

The network runs Nominated Proof-of-Stake. The EVM layer has no separate consensus of its own — it inherits block production and finalisation from the Substrate runtime, with an EVM built on Frontier. [3] The published specification gives a 15-second block time and finality in one to three blocks, roughly 350 transactions per second with the EVM in use and roughly 1,400 without, a 75,000,000 block gas limit and a 0.5 gwei base gas price. [2]

One difference from Ethereum matters when reasoning about the cost of proof submission. Creditcoin prices transactions using the Substrate notion of weight rather than Ethereum's gas accounting, with the results "designed to produce effectively identical" outcomes. [3] In practice this surfaces as gas estimation that does not always report a precompile revert the way an Ethereum node would — a detail the proof worker in Section 5 has to handle explicitly rather than assume away.

Why a credit desk belongs here

Two properties make this venue a fit for the design rather than an arbitrary host.

- **Cross-chain verification is native, not a contract.** Creditcoin's Universal Smart Contracts let a contract "coordinate, interact with, and update smart contracts across multiple blockchains." [1] The verification step is a precompile in the runtime, so a proof is checked by the same node software that validates the block. There is no bridge contract holding a multisig, and no separate validator set to compromise, between the evidence and the contract consuming it.
- **Finality is fast enough to gate on.** With 15-second blocks and finality inside three, a proof submitted when a price round lands is actionable within roughly the same window a conventional oracle push would take. Synchronous verification therefore costs latency measured against attestation lag, not against the chain.
- **The chain's purpose is credit.** Creditcoin exists to carry lending applications, so a repo desk is a first-class use of the venue rather than a general-purpose contract deployed onto indifferent infrastructure.

Everything in this paper executes on **Creditcoin CC3 Testnet**, chain ID 102031, native currency tCTC.

Ethereum mainnet is read-only throughout: the Chainlink feeds are the real mainnet feeds and nothing in this system writes to them. Ethereum Sepolia holds the collateral escrow.

The Attestcoin readability model

The Attestcoin Protocol is the cross-chain layer hosted on Creditcoin. Its documentation splits the problem into two stages: attestation, described as "proactively tracking and reaching consensus on the state changes of source blockchains", and transaction proving, in which a specific transaction containing the data a builder wants is proved on demand. [4]

Attestation

A set of attestors "listen for new source chain blocks, vote on attestations, and store those attestations on-chain." [4] This runs continuously and independently of any application. By the time a contract wants to read something, the block containing it has typically already been attested — the work of agreeing what a source chain's history is has been done ahead of the request, not in response to it.

Attestation is not instantaneous. Coverage trails the source chain's head, so a freshly emitted event is not immediately provable. In this system that lag is visible rather than hidden: the dashboard deliberately renders a market spot reference beside the proved round so that the gap between them is on screen, and the gap is attestation lag.

Proving and verification

Proofs are generated off-chain and checked on-chain — the documentation is explicit that this is a compute trade-off: "to save on-chain compute, we generate proofs off-chain then verify them on-chain," with builders requesting proofs of the transaction containing a target event from a proof generation server. [4] Two proofs travel together:

- A **Merkle inclusion proof** placing the transaction inside a specific source block.
- A **continuity proof** anchoring that block to the attestation record already stored on Creditcoin.

The **block-prover precompile** at `0x0000000000000000000000000000000000000000000000000000000000000000FD2` "verifies merkle and continuity proofs, signaling whether or not the source chain transaction is valid." [4] Because it is a precompile, the check happens inside the transaction that needs it. There is no window in which unverified data sits in contract storage waiting to be validated, because state is only written after the precompile has returned.

ASCBBase, and what an application must implement

An Attestcoin Smart Contract inherits one entry point. `ASCBBase.execute` computes a query id from the chain key, block height and Merkle proof, rejects any id already processed, calls the precompile, and only then hands control to the application's `_processAndEmitEvent`.

```
// @gluwa/asc-contracts · readability/ASCBASE.sol
function execute(
    uint8    action,           // which handler branch to run
    uint64   chainKey,         // which attested source chain
    uint64   blockHeight,      // the attested block
    bytes    calldata encodedTransaction,
    bytes32  merkleRoot,
    INativeQueryVerifier.MerkleProofEntry[] calldata siblings,
    bytes32  lowerEndpointDigest, // continuity proof
    bytes32[] calldata continuityRoots
) external returns (bool success);
```

Deduplication by query id is a protocol-level guarantee, and a load-bearing one: a replayed proof is still a valid proof. Without the check, anyone could resubmit yesterday's price round and roll the book back to it. The applications here add a second, independent guard in the same spirit — round ids must move strictly forward — because the same source transaction can be reached by more than one route.

Inside the handler, proved bytes are decoded with `EvmV1Decoder`, which exposes receipt fields and log entries filtered by event signature. Three checks turn a decoded log into something an application will act on: the source receipt must have succeeded, the emitting address must be the one the contract is bound to, and the log's shape must match what the feed is known to emit.

```
// the decode-and-bind step inside _processAndEmitEvent
EvmV1Decoder.LogEntry[] memory logs =
    EvmV1Decoder.getLogsByEventSignature(receipt, ANSWER_UPDATED_SIGNATURE);
if (logs.length == 0) revert NoMatchingLog();

EvmV1Decoder.LogEntry memory log = logs[0];
if (log.address_ != RESERVE_FEED) revert UntrustedEmitter(log.address_);
if (log.topics.length != 3 || log.data.length != 32) revert MalformedLog();

uint64 roundId = uint64(uint256(log.topics[2]));
if (roundId <= reserveRoundId) revert StaleRound(roundId, reserveRoundId);
```

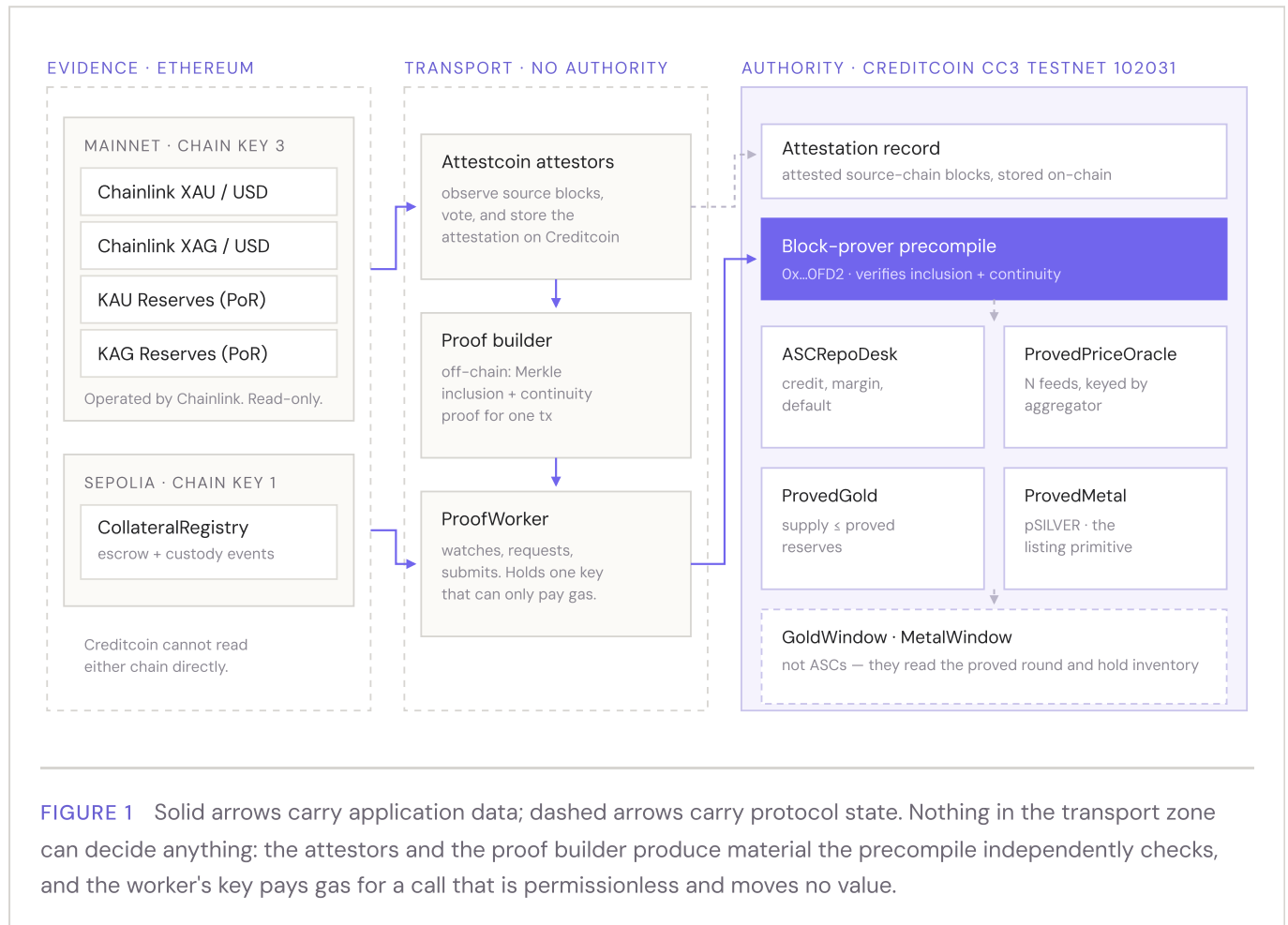
A PRACTICAL BINDING NOTE

Every contract here binds a Chainlink **aggregator**, never the `EACAggregatorProxy` in front of it. The proxy does not emit `AnswerUpdated`; binding it would produce a system that deploys cleanly, verifies proofs correctly, and silently receives nothing forever.

Chain keys carry a similar trap. The published chain list and the live environment have been observed to disagree, so chain keys are read from the chain-info precompile rather than copied from documentation — a wrong key binds the wrong chain and fails only at proof time, long after the code looks correct.

System architecture

Three zones, and the boundary between them is a trust boundary. Ethereum holds the evidence and is never written to. The transport between the chains holds no authority at all. Creditcoin holds every decision that moves value.



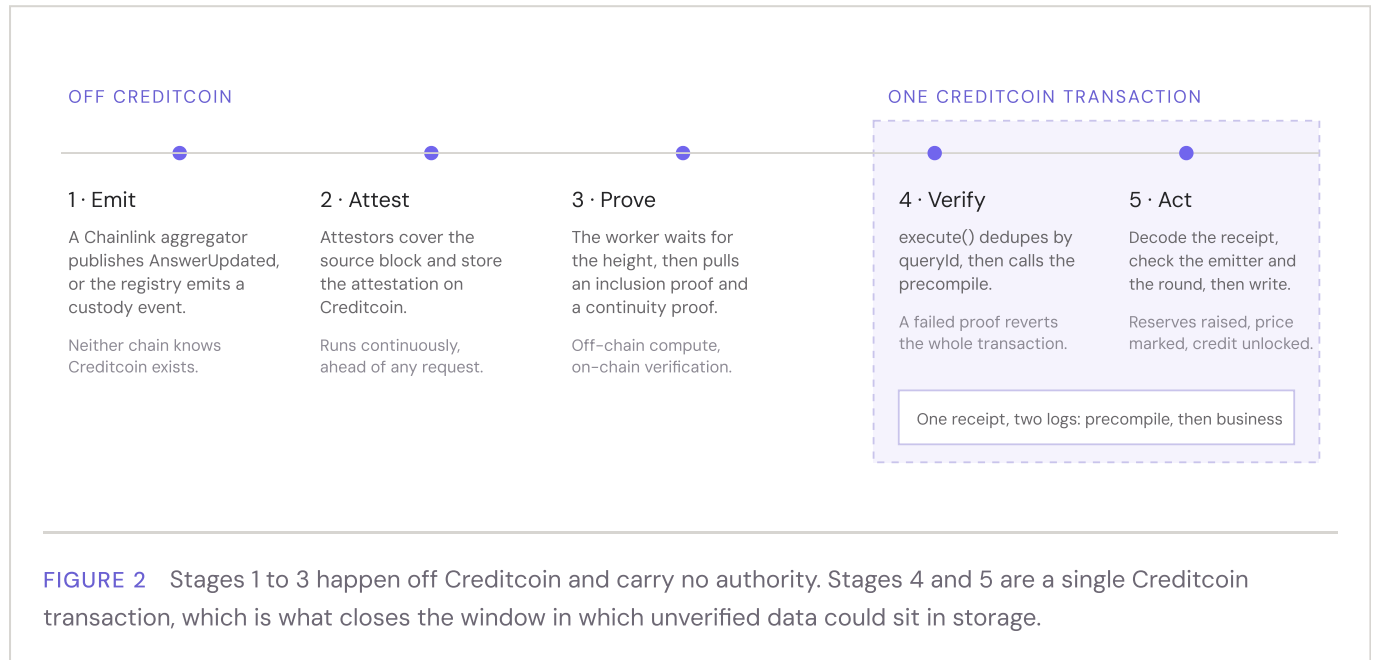
Why the transport holds no authority

The worker is the only component in the system holding a private key, which makes it the obvious thing to attack. It is worth being precise about what compromising it would achieve. `execute` is permissionless, so anyone may submit a proof — the worker is a convenience, not a gatekeeper. It moves no value, so a hostile submitter cannot draw principal, release collateral or mint. A forged proof fails at the precompile, and a replayed one fails at the query-id check. The worst outcome available to a compromised worker is declining to submit proofs, which stalls the desk into its own staleness guards: issuance freezes, windows close, and no bad state is written.

Reads on the other side are similarly narrow. The API returns unsigned calldata for a user's wallet to sign, so the service is not custodial and cannot move a user's position either.

The proof flow, end to end

Five stages carry one fact from a source chain into a decision. The path is identical whether the fact is a metal price, a reserve attestation or a custody event; only the bound emitter and the action discriminator differ.



What the worker does, and what it refuses to do

The worker sweeps each watched chain in windows of fifty blocks, tracking a cursor per watch. For each matching log it records the observation, waits for the source height to be attested, requests the proof, estimates gas and submits. Two behaviours in that loop are worth drawing out because they are consequences of the venue rather than general good practice.

First, **a decodable revert during gas estimation aborts the submission**. A replayed query, a stale round or an unregistered feed will fail however much gas it is given, and submitting anyway only pays for the revert. Second, because pallet-evm does not always surface precompile reverts during estimation, an indeterminate estimation failure falls back to a size-derived gas limit instead of abandoning a proof that would in fact succeed. Treating both failures identically would either burn gas on certain reverts or silently drop valid proofs.

The ledger of observed, attested, proved and confirmed events is held in memory, which is a deliberate scope limit rather than a design claim — see Section 10.

The contract model

Six contracts, of which four are Attestcoin Smart Contracts extending `ASCBase`. The dealing windows are ordinary contracts: they consume a proved round through the oracle's view interface but verify nothing themselves, which keeps the verification surface as small as the set of contracts that actually need it.

CONTRACT	CHAIN	BASE	ROLE
ASCRRepoDesk	Creditcoin	ASCBase	Credit agreements: terms, proved pledge, draw, margin, default. Reads two source chains in four action branches.
ProvedGold	Creditcoin	ASCBase + ERC-20	pGOLD. Supply capped by a proved KAU Reserves round. Feed bound once, after deployment.
ProvedMetal	Creditcoin	ASCBase + ERC-20	The listing primitive. Name, symbol and reserve aggregator are constructor arguments; pSILVER is one instance.
ProvedPriceOracle	Creditcoin	ASCBase	One contract, many feeds. The emitting aggregator identifies the feed, so listing a price source is a transaction.
MetalWindow	Creditcoin	plain	Two-way dealing window. Fills at the last proved round; holds inventory, so it can never mint.
CollateralRegistry	Sepolia	plain	Source-chain escrow. Holds collateral, emits unambiguous events, and knows nothing about Creditcoin.

Supply bound to reserves

The issuance guard is the clearest statement of the whole design, and it is four lines long. `provedReserves` moves only inside `_processAndEmitEvent`, which runs only after the precompile has verified a round from the bound Proof-of-Reserve aggregator.

```
// contracts/sol/ProvedMetal.sol
function issue(address to, uint256 units) external onlyOwner {
    if (provedReserves == 0) revert NoReservesProved();
    if (block.timestamp > reserveUpdatedAt + MAX_RESERVE_AGE) revert StaleReserves(reserveUpdatedAt);

    uint256 available = headroom();
    if (units > available) revert ExceedsProvedReserves(units, available);

    _mint(to, units);
}
```

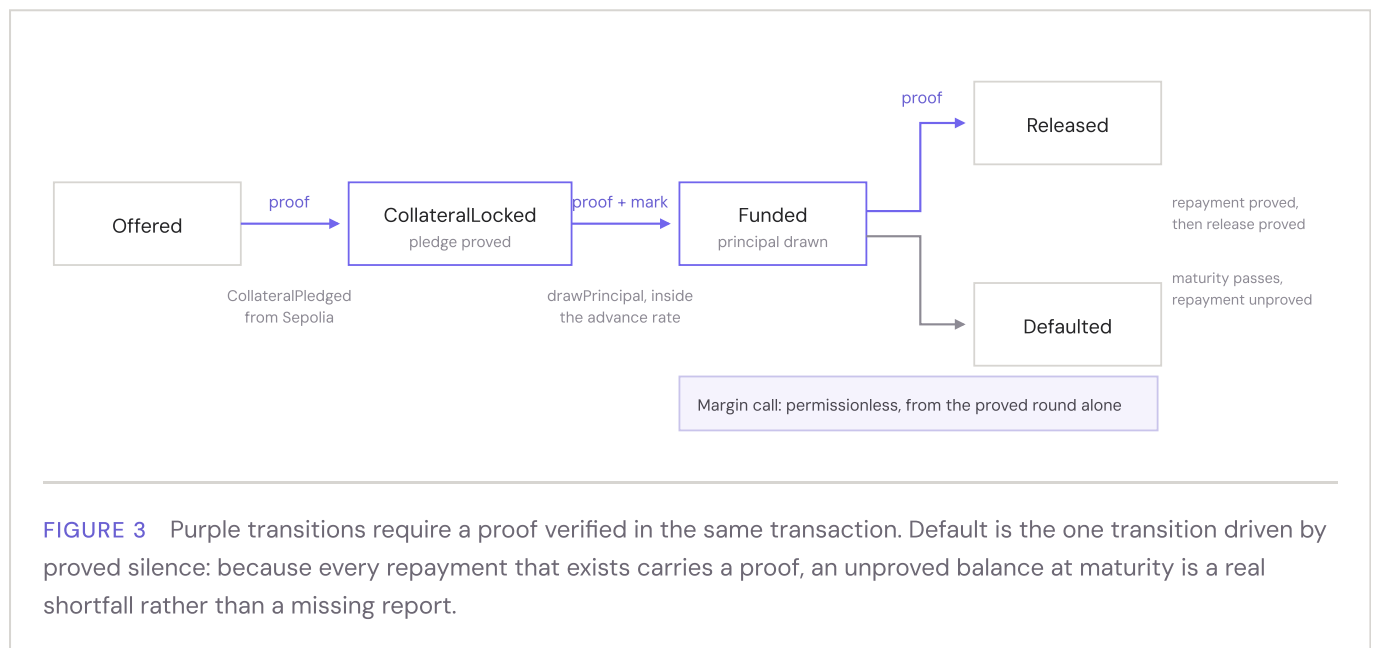
The issuer holds `onlyOwner` and can therefore choose to mint less, never more. The ceiling is a figure they do not control and cannot forge, and with no proof at all the token cannot be issued even once.

Units are matched on purpose

Tokens are denominated in the reserve feed's own unit — grams for both listed metals — so the safety-critical comparison `totalSupply() + amount <= provedReserves` needs no unit conversion. Gram-to-troy-ounce arithmetic exists only in the pricing path inside `MetalWindow`, using a troy ounce of 31.1034768 g scaled by 10^7 to stay in integer arithmetic. The trade-off is deliberate: a rounding error in pricing costs basis points on a fill, whereas a rounding error in the supply cap breaks the invariant the token exists to enforce.

The credit lifecycle

`ASCRepoDesk` moves an agreement through five states. Two transitions are gated on proofs from a source chain, one on a proof plus a mark-to-market check, and two on conditions the chain can evaluate for itself.



Both risk actions are permissionless. `markUndercollateralised` succeeds only if the last proved round puts coverage below the maintenance margin, and reverts with `CoverageAdequate` otherwise; `clearMarginCall` lifts the flag once a later proved round restores it. No privileged role decides either, because both follow deterministically from proved state.

The refusals

The errors below are the enforcement surface. Each is a named revert rather than a guard that degrades to a default.

CONTRACT	ERROR	REFUSES
ProvedGold / ProvedMetal	NoReservesProved()	Issuance before any reserve round has been proved.
ProvedGold / ProvedMetal	ExceedsProvedReserves(requested , headroom)	Issuance beyond the proved reserve figure.
ProvedGold / ProvedMetal	StaleReserves(updatedAt)	The last proved reserve round is older than 26 hours.
all ASCs	UntrustedEmitter(emitter)	The proved log was emitted by an address the contract is not bound to.
all ASCs	StaleRound(proved, stored)	A valid proof of a round at or behind the stored one — a replay.
ASCRepoDesk	UnexpectedStatus(agreementId, status)	A lifecycle step taken out of order, including drawing before the pledge is proved.
ASCRepoDesk	StalePrice(updatedAt)	Mark-to-market against a price older than 26 hours.
ASCRepoDesk	ExceedsAdvanceRate(principalUsd , maxUsd)	A draw that would exceed the advance rate at the proved mark.
ProvedPriceOracle	UnknownAggregator(aggregator)	A proved round from a feed that was never registered.

Listing a real-world asset

Silver was listed after gold without a line of new contract code. That is the test of whether the model generalises, and the path has four steps.

1. **The asset needs a Proof-of-Reserve feed on an attested chain.** This is the gate, and it is deliberately the hardest step. The feed is trustworthy precisely because the issuer does not operate it; an issuer who cannot obtain independent reserve attestation cannot list here, which is the intended outcome rather than a limitation to engineer around.
2. **Deploy `ProvedMetal`** with its name, symbol and that aggregator. The feed is immutable in the constructor, so supply is capped by proved reserves permanently and irrevocably from the first block.
3. **Register its price feed** with `registerAggregator(...)` on the existing `ProvedPriceOracle` — one transaction, no deployment. Because a proved round is keyed by its emitting aggregator, one oracle serves every listed asset.
4. **Deploy a `MetalWindow`** against that aggregator to make the asset tradeable, and call `registerCollateralAsset(...)` on the desk with its advance rate and maintenance margin to make it borrowable.

Adding the feeds to the worker's watch list and the asset to the dashboard's listings is configuration, held as JSON in the deployment environment rather than in code. The worker picks up the new aggregator on restart and begins proving its rounds.

WHAT GENERALISES, AND WHAT DOES NOT

The listing path is asset-agnostic in its mechanics but not in its units. A metal denominated in grams against a feed that reports grams needs no conversion in the supply cap; an asset whose reserve feed reports in a different unit from its token would reintroduce arithmetic into the safety-critical path. The honest generalisation is therefore: any asset with an independent Proof-of-Reserve feed can be listed, provided its token is denominated in that feed's unit.

Risk parameters

Every value below was read from the live testnet deployment rather than from configuration files, since a deployed parameter and an intended one are not the same claim.

PARAMETER	VALUE	WHERE	EFFECT
Advance rate	6,000 bp (60%)	<code>ASCR RepoDesk.collateralAssets</code>	Ceiling on principal against proved collateral value at draw.
Maintenance margin	7,500 bp (75%)	<code>ASCR RepoDesk.collateralAssets</code>	Below this coverage a funded agreement can be margin-called by anyone.
Max price age	93,600 s (26 h)	<code>ASCR RepoDesk.MAX_PRICE_AGE</code>	Chainlink XAU/USD heartbeat is 24 h; the guard adds two hours of slack.
Max reserve age	93,600 s (26 h)	<code>ProvedMetal.MAX_RESERVE_AGE</code>	Past it, issuance freezes rather than trusting an old reserve figure.
Dealing spread	50 bp	<code>MetalWindow.spreadBps</code>	Applied symmetrically around the proved mid. Capped at 1,000 bp by the constructor.
Price decimals	8	<code>ASCR RepoDesk.PRICE_DECIMALS</code>	Matches the Chainlink metal feeds.

How the advance rate is enforced

At `drawPrincipal`, collateral value is computed from the pledged amount and the last proved round, the advance rate caps the permitted principal, and a draw above that cap reverts. The realised loan-to-value is computed on-chain and emitted with the draw, so it is an observed figure rather than a reported one:

```
ltvBps = (principalUsd * 10_000) / collateralValueUsd
```

In the worked example of Section 9, \$20,000 was drawn against ten troy ounces marked at \$4,349.135 — \$43,491.35 of bullion — giving 4,598 bp, or 45.98%, comfortably inside the 6,000 bp advance rate.

On the choice of 26 hours

Both metal price feeds and both Proof-of-Reserve feeds publish on a 24-hour heartbeat. A staleness bound equal to the heartbeat would trip on ordinary jitter; two hours of slack absorbs that without materially widening the exposure. The bound is a hard stop in both directions — past it, issuance reverts with `StaleReserves` and dealing windows close with `StalePrice` rather than filling on a figure nobody has re-proved.

Evidence from the testnet deployment

The claim that proofs are preconditions rather than decoration is testable, and the test is that the same call fails before the proof and succeeds after it. Both halves are recorded below on Creditcoin CC3 Testnet.

The refusals

ATTEMPTED	RESULT
issue before any reserve proof	reverts <code>NoReservesProved()</code>
<code>drawPrincipal</code> before the pledge is proved	reverts <code>UnexpectedStatus(..., 1) · selector 0xaf4cf409</code>
issue one unit past proved headroom	reverts <code>ExceedsProvedReserves(2566134..., 2566133466...)</code>

Then the successes

STEP	TRANSACTION
Gold price proved from mainnet <code>GoldPriceProved(434913500000, 10212, ...)</code>	<code>0x7dbfd096...c855d</code>
Pledge proved from Sepolia <code>CollateralProved(..., 10 oz, ...)</code>	<code>0xce551f35...1e4a</code>
drawPrincipal, after both proofs <code>PrincipalDrawn(..., 40 tCTC, 4598)</code>	<code>0xbdae44c9...7e3e</code>

Both assets, on real feeds

ASSET	PRICE FEED	RESERVE FEED	PROVED VALUE	FILL
pGOLD	XAU / USD	KAU Reserves	\$4,349.135 / oz 2,567,133.466 g	<code>0x8f3ebb0c...c18d</code> 10 g, \$1,405.270767, round 10212
pSILVER	XAG / USD	KAG Reserves	\$64.486 / oz 3,688,827.985 g	<code>0xbb122fab...aa5d</code> 100 g, \$208.363940, round 29850

Every fill emits the Chainlink round id it priced against, so a trader can check their execution against a specific proved round rather than against a figure the desk asserts after the fact. The proved reserve figures above were read back from the deployed tokens at the time of writing.

The contract suite carries 55 Solidity tests and the API and domain layer 17, covering the refusals above alongside replay rejection, untrusted-emitter rejection and the lifecycle transitions.

What this design does not prove

This section is the one to read adversarially. A proof system oversold is worse than one honestly bounded, because the overselling is what makes people stop checking.

- **It proves provenance, not truth.**

The system proves that a named Chainlink aggregator published a particular number in a particular block. It does not prove the number is correct. What it removes from the trust path is the relay and the issuer's self-report; the auditor behind a Proof-of-Reserve feed remains trusted, as does Chainlink's operation of the feed. Anyone describing this as trustless reserve verification is overstating it.

- **The demonstration borrows someone else's reserve feeds.**

`KAU Reserves` and `KAG Reserves` attest Kinesis's vaults. The listings therefore model an issuer whose reserves those feeds report — a stand-in, in the same way `TestBullion` stands in for a production bullion token. In a real deployment an issuer commissions their own Proof-of-Reserve feed and only the bound aggregator address changes; no contract logic does.

- **There is no CTC/USD Chainlink feed.**

Quoting the dealing windows in a USD token is forced rather than chosen: no provable way to price metal in tCTC exists today. Credit is drawn in tCTC, but the mark and the quote are USD-denominated, and closing that gap needs a feed that does not currently exist.

- **The direction is read-only.**

Attestcoin's writability path was in third-party audit at the time of writing, so Creditcoin never writes to Ethereum. Releasing source-chain collateral is a source-chain decision whose event is then proved back to Creditcoin — the desk observes the release, it does not command it.

- **Trust binds to an emitting address, not to a chain.**

`ASCBASE.execute` does not pass `chainKey` through to the handler, so each contract checks that a proved log came from the address it is bound to, following the reference examples. A bound address on a different attested chain would satisfy that check. This is a known property of the current base contract, not a mitigation the applications implement.

- **Prices are only as fresh as the last proved round.**

A sharp move between rounds is invisible to the desk until the next round is proved. The 26-hour guard bounds how stale a mark can be before the system stops acting on it; it does not make the mark continuous, and a lender should read the advance rate as compensation for that discreteness.

- **Operational scope is a demonstration's scope.**

Exactly one worker may run — two would prove the same events twice and pay for the duplicate's revert. Watches sweep sequentially, so a large backfill on one chain delays the others. The proof ledger is held in memory and does not survive a restart; it is an observability aid, and no contract state depends on it.

None of these undermine the three claims of Section 1, and it is worth being precise about why. Each claim is of the form "the contract will not act unless a proof of the form it expects verifies in the same transaction."

That property holds under every limit above. What the limits bound is the meaning of the proved fact — whose vault, whose audit, how fresh — rather than the enforcement of the gate.

Deployment reference

Creditcoin CC3 Testnet · chain 102031

CONTRACT	ADDRESS
ASCRRepoDesk	0x0231762F2F2285F6ea27Ad456E144C1371e4AF3B
ProvedGold (pGOLD)	0x2a8142Db4C3b9033339A6E25b225e808098BDB0
ProvedPriceOracle	0x9B6eB52D26CeF7b04bb52b307Db86262bD8D6C8A
ProvedMetal (pSILVER)	0x74f6E83aA79a16CeD41Ec5b0697879E51e81cAd0
GoldWindow	0x2369B00a916132cBD3639bB29353d062f5fF325a
MetalWindow (silver)	0x2BE77D62F16438ce44c41120846321d871b73806
TestUSD (tUSD)	0x63b8493f7508Bea15b5D29A369E74D99Ac8932d6

Ethereum Sepolia · chain key 1

CONTRACT	ADDRESS
CollateralRegistry	0x3A053Dbfffb16C033eF16eBcD4dE45673BE3346d4
TestBullion (tXAU)	0x626DA908bdE6F66f9178B3128dd17D82dc74CE21

Ethereum Mainnet · chain key 3 · read-only

Aggregator addresses, not proxies — the proxy does not emit `AnswerUpdated` .

FEED	ROLE	AGGREGATOR
XAU / USD	price	0x0e3dd634FFbF7EA89BbDCF09Ccc463302FD5f903
XAG / USD	price	0xB38d1D12Ba17aA62255e588a0bC845c1a589A50d
KAU Reserves	proof of reserve	0x9b3a984d1abbe03845CBa7A895f1ff7f4209d59c
KAG Reserves	proof of reserve	0x3B4f49f4aa5491B5a60C0724467A67b4910aEAAc

◆ REFERENCES

1. Creditcoin documentation, What is Creditcoin. <https://docs.creditcoin.org/>
2. Creditcoin documentation, Technical Specification. <https://docs.creditcoin.org/technical-specification>
3. Creditcoin documentation, EVM-compatibility. <https://docs.creditcoin.org/evm-compatibility>
4. Attestcoin Protocol documentation, Attestcoin Readability. <https://docs.attestcoin.org/attestcoin-protocol/attestcoin-readability>
5. Attestcoin Protocol documentation. <https://docs.attestcoin.org/>
6. Chainlink. <https://chain.link/>
7. Chainlink reference data directory, Ethereum mainnet feeds. <https://reference-data-directory.vercel.app/feeds-mainnet.json>
8. Creditcoin Testnet block explorer. <https://creditcoin-testnet.blockscout.com>